AI Capability: Compute or Efficiency?

DVN36104 Data Visualisation and Narratives — Assignment 3
Group 18

Deployed app: https://dvn-at3-group18.streamlit.app/

Repository: https://github.com/chloetr7/dvn_at3_group18

Assessed version: [ADD FINAL COMMIT HASH OR RELEASE TAG AFTER THE GROUP FREEZES THE SUBMISSION VERSION]

> The assessed version identifies the exact GitHub state that should be marked. Example: commit `a1b2c3d` or release tag `v1.0-final`.

Project Question

To what extent have recent frontier AI capability gains depended on increasing training compute versus improvements in compute efficiency?

This project investigates whether recent improvements in frontier AI capability are primarily associated with larger training-compute budgets, or whether newer models also appear to achieve stronger capability at comparable levels of reported training compute.

Target Persona

Our target persona is an **AI Infrastructure Lead at a mid-sized technology company** who needs to decide how much to invest in increasingly compute-intensive models versus more compute-efficient alternatives.

The dashboard is designed to help this user understand:

- how quickly AI training compute has grown;
- how strongly training compute is associated with measured capability;
- whether newer models achieve stronger capability at comparable compute levels; and
- whether models capable of meeting a given performance requirement can be found with lower reported training compute.

Answer in Brief

Across the Epoch AI data used in this project, training compute has increased substantially over time and is positively associated with GPQA Diamond performance.

However, compute does not explain the full variation in measured capability. Models trained within similar compute ranges can achieve substantially different GPQA Diamond scores, and newer models in some comparable-compute ranges achieve higher measured performance than earlier models.

These patterns are consistent with improvements in compute efficiency, but the observational data do not establish that algorithmic progress alone caused the differences. Architecture, training data, post-training methods, model selection, inference-time techniques and missing compute disclosures may also contribute.

Macro Overview — Compute Growth

Dashboard section: `Overview · Compute growth`

Objective

This section sets the scene for the user stories. Before asking what training compute buys, it shows the AI Infrastructure Lead how quickly the compute behind notable AI models has grown, and therefore how fast the cost of staying at the frontier is rising.

Data and methodology

The analysis uses `data/ai_models.csv`, prepared from Epoch AI — Notable AI Models. It contains **351** models released from 2018 to 2026, all with positive reported training compute and parameter counts. Missing values are excluded rather than imputed.

Growth is estimated with an ordinary least-squares fit of $\log_{10}$(training FLOP) on release date. The slope is converted into a growth multiple per year and a doubling time. Yearly medians are calculated separately for Open-Weight and Closed / API-Only models, and are shown only where a year and access group has at least **three** models.

Visualisation design

The main visual is a **scatter plot of every model's training compute over time on a logarithmic axis**, with a fitted trend line and labels for the largest training run of recent

years.

A linear axis was considered but rejected: training compute spans more than ten orders of magnitude, so the largest recent runs would flatten almost every earlier model against the floor and hide the exponential trend. Showing yearly averages only was also rejected as the main view, because individual models are what the persona recognises and can inspect.

A second line chart compares the yearly median compute of Open-Weight and Closed / API-Only models.

Interactions

Users can filter by release-year range, model access group and primary domain, and choose which models are labelled. The KPIs, trend line, charts, written takeaway and verification output update with the filters.

Key finding

Across all 351 models, training compute grew by about **×5.7 per year**, doubling roughly every **4.8 months**. Over the 8.6 years covered, this compounds to about 3.3 million times. The trend explains about 53% of the variation in log-compute (R^2).

Since 2024, the yearly median compute of Open-Weight models is similar to that of Closed / API-Only models. This should be read with caution, because many frontier closed models do not disclose their training compute and are therefore missing from the data.

Verification

The section includes a visible verification panel with two checks:

- **Yearly median check:** the plotted yearly median is recalculated from the underlying models' FLOP values. For example, the 2026 Open-Weight median of **4.32×10^{24} FLOP** is recalculated from its **17** underlying models and matches the displayed value.
- **Source trace:** any displayed model can be looked up in the original `data/raw_ai_models.csv` export by model name and publication date. The app confirms that training FLOP and parameter count match the raw Epoch AI record exactly.

Limitations

Epoch AI lists *notable* models, and only models with published compute and parameters are kept, so the largest closed runs may be under-represented. Many FLOP values are Epoch AI estimates, so the trend is more reliable than any single point. The Open-Weight / Closed grouping is a project-derived access category, not a legal licence. The analysis is

descriptive and does not establish what caused compute growth or whether compute caused capability gains.

User Stories

User Story 1 — Compute vs Capability

As an AI Infrastructure Lead, I want to compare measured AI capability with training compute, so that I can assess whether higher-compute models are consistently associated with stronger performance.

Dashboard section: `US1 · Compute vs capability`

Objective

This section tests the simplest version of the "scale" explanation. If capability depended mainly on training compute, higher-compute models should score consistently higher. The AI Infrastructure Lead needs to know not only *whether* compute and capability move together, but whether they do so consistently enough to choose a model on compute alone.

Data and methodology

The analysis uses `data/gpqa.csv` (Epoch AI Capabilities & Benchmarking, GPQA Diamond only). Of **316** GPQA Diamond results, the **93** with disclosed positive training compute are used; the rest are excluded, not imputed.

- An ordinary least-squares fit of GPQA score (percentage points) on $\log_{10}$(training FLOP) gives the average change per 10× compute, and its R^2 shows how much of the variation compute accounts for.
- A Spearman rank correlation measures whether the association is monotonic, without assuming a straight line.
- Results are grouped into one-order-of-magnitude compute bands. The score range within each band (bands with at least **five** results) measures how far apart models with comparable compute can be.
- Residuals from the trend identify the models furthest above and below what compute alone would suggest.
- Some models were evaluated in several versions that share one compute estimate. A *Best version per model* option keeps one point per model (72 points) to check that repeated models do not drive the result.

Visualisation design

The main visual is a **scatter plot of GPQA Diamond score against training compute on a logarithmic axis**, with the fitted trend line. Points can be coloured by release year or by model access, and the largest deviations from trend are labelled with their score.

A linear compute axis was rejected because compute spans several orders of magnitude, which would compress most models into one corner. A bar chart of average score per compute band was also rejected as the main view: averages would hide exactly the spread this user story asks about.

A second chart, a **box plot of scores within each compute band** with every result shown as a point and each band's median printed on the chart, makes the consistency question explicit.

Interactions

Users can filter by release-year range and model access group, switch the unit of analysis (all evaluated versions or best version per model), colour points by release year or access, and choose how many deviations from trend to label. The KPIs, written takeaway, both charts, deviation tables and verification output update with these selections.

Key finding

Higher training compute is associated with higher GPQA Diamond scores: about **+14.8 percentage points per 10× compute** (Spearman $\rho = 0.65$, $n = 93$). The association is not consistent, though. The trend explains only **36%** of the variation in scores (R^2), and in the 10^{24}–10^{25} FLOP band scores range from **27% to 92%**, a 65-point gap between models with comparable compute. The pattern holds with one point per model (+13.8 pp per 10×, $\rho = 0.62$, $R^2 = 31\%$, $n = 72$).

For the AI Infrastructure Lead, compute alone is a weak basis for choosing a model. This motivates User Story 2, which asks whether newer models achieve more at comparable compute.

Verification

The section includes a visible verification panel with two checks:

- **Compute-band check:** for a selected band, the app lists every underlying result and confirms that the plotted box spans exactly the lowest and highest measured scores. For example, the 10^{24}–10^{25} FLOP band contains **45** results ranging from **26.96% to 91.67%**, with a median of **65.34%**.

- **Source trace:** any displayed point can be looked up in `data/raw_benchmarks.csv` by its benchmark run ID. For example, DeepSeek V4 Flash 0731 (run `EDFusDaaFkh5n2MqKNxMBD`) has a raw score of 0.910354 and 2.496 × 10^{24} FLOP; the app shows 91.04% and the same FLOP, and confirms the match.

Limitations

The analysis is an association, not a causal estimate. Only pre-training compute is counted, so post-training, reinforcement learning and inference-time reasoning can place newer models above the trend. Closed frontier models rarely disclose compute and are under-represented (17 of 93 results). Many FLOP values are Epoch AI estimates. GPQA Diamond tops out near 100%, which compresses differences among the strongest models.

User Story 2 — Capability at Comparable Compute

As an AI Infrastructure Lead, I want to compare the capability of models released at different times under similar training-compute levels, so that I can assess whether newer models are achieving better performance without proportionally increasing compute.

Dashboard section: `US2 · Capability at comparable compute`

Analytical approach

This section uses measured **GPQA Diamond** benchmark observations from Epoch AI.

The preparation process:

1. Starts from the Epoch AI benchmark export.
2. Filters observations to **GPQA Diamond**.
3. Standardises benchmark scores as percentages.
4. Retains observations with reported positive pre-training compute.
5. Excludes missing training-compute observations rather than estimating or imputing them.
6. Groups observations into broad one-order-of-magnitude compute bands.
7. Compares yearly GPQA distributions only where at least three measured observations are available for a release year.

The default view uses the **10^{24}–10^{25} FLOP** training-compute range.

Current finding

Within the default 10^{24}–10^{25} FLOP range:

- **45** measured observations are included;
- **3** release years contain enough observations to be displayed;
- overall median GPQA Diamond performance is **65.3%**;
- yearly median GPQA increases from **40.7% in 2024** to **89.7% in 2026**, an increase of approximately **49.1 percentage points**.

This result is consistent with capability improvement at broadly comparable reported training-compute levels.

It should **not** be interpreted as causal evidence that algorithmic progress produced the improvement. Differences may also reflect architecture, training data, post-training methods, reasoning techniques and model selection.

Visual design

The primary visual is a **line chart of yearly median GPQA Diamond performance**, with interquartile-range error bars.

The analytical task is a comparison of capability over time while constraining training compute to a comparable range.

A scatter plot of all individual observations was considered as an alternative. It was not selected as the primary visual because the main task is to communicate the year-to-year pattern rather than individual-model variation. With many observations, a scatter plot would make the temporal comparison less direct.

The median provides a clearer summary of the yearly pattern, while the interquartile range preserves information about within-year variation.

A release year is shown only when at least **three measured observations** are available.

Interaction

Users can select one of the available training-compute bands:

- 10^{23}–10^{24} FLOP
- 10^{24}–10^{25} FLOP
- 10^{25}–10^{26} FLOP

The KPIs, yearly summary, chart and verification output update to match the selected range.

Verification and traceability

A displayed yearly median can be traced through the following data lineage:

`Epoch AI benchmark export`
→ `raw_benchmarks.csv`
→ `prepared_data.py`
→ `benchmarks.csv`
→ GPQA Diamond filter
→ `gpqa.csv`
→ selected compute-band filter
→ yearly median

For example, in the default compute range, the displayed **2026 median GPQA score is 89.74%**, based on **10 measured observations**.

The application independently:

- recalculates the median from the underlying observations;
- verifies that it matches the displayed median;
- verifies that every included observation falls inside the selected compute band; and
- verifies that the displayed observation count matches the underlying rows.

The individual observations used in the calculation can also be inspected directly in the application.

Limitations

This analysis controls compute only within a broad order-of-magnitude range, rather than comparing models with exactly equal training FLOP.

It also cannot establish a causal estimate of algorithmic efficiency. Differences between models may reflect architecture, training data, post-training methods, model selection and other factors.

Observations without published training compute are excluded, which may introduce selection bias.

No benchmark scores or compute values are imputed.

User Story 3 — Resource-Efficient Alternatives

As an AI Infrastructure Lead, I want to identify models that achieve strong measured capability with comparatively lower training compute, so that I can evaluate more resource-efficient alternatives instead of selecting models based on scale alone.

This user story is explored through two complementary views:

User Story 3A — Compute required to reach a capability level

Objective

This view helps an AI Infrastructure Lead judge whether a cheaper-to-train alternative is likely to exist for the capability they need. The user picks a GPQA Diamond level, and the tab shows which model reached it first, which model has since reached it with the **least reported training compute**, and how long that took.

Data and Methodology

The analysis uses `data/gpqa.csv` (Epoch AI — Capabilities & Benchmarking). Of the 316 GPQA Diamond results, 93 have positive reported training compute; these cover 72 models released between March 2023 and September 2026. Missing compute is excluded, not imputed.

Each model is represented by its best-scoring GPQA run (the same rule as User Story 1), because different evaluated versions of one model share one training-compute value. For a chosen level *L*:

- qualifying models are those scoring at least *L*, ordered by release date;
- a model sets a **record** when its training compute is lower than that of every earlier qualifying model; and
- the **reduction factor** is the compute of the first model to reach *L* divided by the lowest compute of any model that has reached *L*.

Levels start at 40% because random guessing on GPQA Diamond scores 25%.

Visualisation Design

- **Chart 1 (change over time):** every model's training compute (log scale) against release date, with a green step line showing the lowest compute that has reached the chosen level so far. Record-setting models are labelled with their name, compute and score, so values can be read without hovering.
- **Chart 2 (magnitude):** a dumbbell chart on a log scale comparing, for each level from 40% to 95%, the compute of the first model to reach it with the least compute that has reached it since. The length of each bar is the size of the reduction.

A grouped bar chart was considered for chart 2 but rejected: compute spans roughly 10^{23} to 10^{27} FLOP, so on a linear axis the smaller models' bars would be invisible, and bars on a log axis do not start at zero, so their length would mislead. The dumbbell avoids both problems.

Colours follow the colour-blind-safe Okabe-Ito palette used across the app, and open-weight and closed models also differ by marker shape.

Interactions

- **Capability-level slider** (40–95% GPQA Diamond): updates the KPIs, the insight box, chart 1 and the verification list.
- **Access filter** (all models / open-weight only / closed or API-only): updates every view, including chart 2.
- If no model reaches the chosen level under the filter (for example, an open-weight model at 95%), the tab shows a message explaining why instead of an empty chart.

Key Finding

At the default level of 80%, **Grok 4** first reached it in July 2025 with 5.0×10^{26} FLOP. **DeepSeek V4 Flash 0731** reached the same level 13 months later with 2.5×10^{24} FLOP: about **200 times less training compute**.

Across the seven levels compared (all models), a later model reached the same score with less compute at six levels, between 3.9× and 200× less, with a median gap of about 10 months. The highest level (≥ 95%), however, has only been reached by GPT-6 Astra, with 9.4×10^{26} FLOP. In this data, **new top capability arrives with large compute first, and efficiency follows**.

These are observed associations among models with published compute, not evidence of why a later model needed less compute.

Verification

The section "3 · Verify a displayed value" traces any record-setting model from the tab back to `data/raw_benchmarks.csv` by benchmark run ID, and checks that the raw score × 100 equals the displayed score and that the training FLOP is identical.

For example, Grok 4 (run ID `A85Zfq2qguE4X9xXBweBHP`) has a raw score of 0.87 and training compute of 5×10^{26} FLOP in `raw_benchmarks.csv`, matching the displayed 87% and 5.0×10^{26} FLOP. Dividing its compute by DeepSeek V4 Flash 0731's raw value (2.496×10^{24} FLOP) gives 200.3, which matches the displayed reduction.

Limitations

- The analysis cannot show *why* a model needed less compute (better data, distillation, architecture or post-training).
- Distilled models report only their own training FLOP, not the compute of the larger model

they learned from.

- Training compute says nothing about inference cost, latency or price.
- Many GPQA results, often from closed labs, have no published compute, so some levels may have been reached earlier or more cheaply than shown.
- GPQA Diamond measures graduate-level science questions only, not all capabilities.

User Story 3B — Resource-efficient model choices

Objective

This dashboard supports an AI Infrastructure Lead in identifying models that achieve strong measured capability with comparatively lower training compute.

Data and Methodology

The analysis uses the Epoch AI GPQA Diamond benchmark dataset. Among 316 observations, 93 have reported positive training compute and are eligible for comparison. Missing compute values are excluded rather than imputed.

Visualisation Design

A ranked dot plot with a logarithmic horizontal axis is used to compare the training compute of qualifying models.

A standard linear bar chart was considered but rejected because training compute varies by several orders of magnitude, making smaller values difficult to compare.

The dashboard also identifies Pareto-efficient candidates based on benchmark performance and training compute.

Interactions

Users can adjust the minimum GPQA Diamond score, release-year range, organisation and number of models displayed. The ranking and summary metrics update based on these selections.

Key Finding

With the default minimum GPQA Diamond score of 60%, 43 observations qualify. Among these observations, gpt-oss-20b has the lowest reported training compute of approximately 5.49×10^{23} FLOP, with a measured GPQA Diamond score of 60.80%.

This finding applies to the selected observations and performance threshold, rather than

identifying the universally most efficient AI model.

Verification

The dashboard traces displayed values through the original Epoch AI benchmark export, `prepared_data.py`, `benchmarks.csv` and `gpqa.csv`.

For example, the original GPQA Diamond score for gpt-oss-20b is 0.60795455. Multiplying it by 100 and rounding produces the displayed score of 60.80%. Its reported training compute also matches the original record.

Limitations

Training compute does not directly measure inference cost, latency or overall algorithmic efficiency. Missing compute disclosures may affect the ranking. The observed relationships do not establish causality.

Dashboard Structure

The application contains five analytical sections:

Section	Purpose
Overview · Compute growth	Establish how rapidly reported AI training compute has increased
US1 · Compute vs capability	Examine the relationship between training compute and measured GPQA capability
US2 · Capability at comparable compute	Compare capability across release years while constraining compute
US3A · Compute to reach a level	Examine whether a given capability level has later been reached with less compute
US3B · Efficient model choices	Identify measured models meeting a selected capability requirement with comparatively lower compute

---

Data Sources

Epoch AI — Notable AI Models

Used primarily for the macro overview of AI training-compute growth.

Source:
https://epoch.ai/data/ai-models

Epoch AI — Capabilities & Benchmarking

Used for the GPQA Diamond capability analyses.

Source:
https://epoch.ai/benchmarks/use-this-data

Licence / attribution: **CC BY**

Full source fields, units, transformations and coverage are documented in:

`data/DATA_DICTIONARY.md`

Data Processing

Raw source files are stored in the `data/` directory.

The processing pipeline is implemented in:

`prepared_data.py`

Key processed files include:

- `data/ai_models.csv`
- `data/benchmarks.csv`
- `data/gpqa.csv`

The processed datasets are generated from the documented source files rather than manually edited dashboard values.

To regenerate the processed datasets:

```
python prepared_data.py
```

No command-line arguments are required. The script also writes `data/metadata.json`, which records when the processed files were regenerated and the date coverage of each dataset; `app.py` reads it to show **Data coverage** and **Last refreshed** on the first screen.

Running the Application Locally

Install the required Python packages:

```
pip install -r requirements.txt
```

Then run:

```
streamlit run app.py
```

Repository Structure

```
.
├── app.py
├── prepared_data.py
├── requirements.txt
├── README.md
│
├── tabs/
│   ├── tab1.py
│   ├── tab2.py
│   ├── tab3.py
│   ├── tab4.py
│   └── tab5.py
│
├── data/
│   ├── DATA_DICTIONARY.md
│   ├── raw_ai_models.csv
│   ├── raw_benchmarks.csv
│   ├── ai_models.csv
│   ├── benchmarks.csv
│   └── gpqa.csv
│
└── AI_DISCLOSURE.md
```

Adjust this tree if the final repository structure differs.

Verification and Reproducibility

Displayed analytical results are calculated from the processed datasets rather than manually entered.

Each analytical section includes a verification mechanism that allows selected displayed values to be traced back to their underlying observations and, where applicable, to the raw Epoch AI source export.

The data dictionary documents source provenance, variables, units, time coverage and transformations.

Data and AI Disclosure

Generative AI tools were used as development assistants during the project.

Uses included activities such as debugging code, reviewing Streamlit layout and presentation structure, and refining written explanations.

AI-generated suggestions were not accepted without verification. Team members checked relevant outputs against source data, executed the code, inspected underlying observations and compared displayed calculations with independently recalculated values.

A detailed record of substantive AI use and the verification performed by each team member is provided in:

`AI_DISCLOSURE.md` (AI Provenance Matrix section)

A full Data & AI Disclosure is provided in:

`AI_DISCLOSURE.md`

Team Contributions

Member	Main contribution
Yuchao Zong	Macro overview (Tab 1): training-compute growth analysis, visualisation, verification and README section
Qi Wang	User Story 1 (Tab 2): compute-vs-capability analysis, visualisation and verification; main app (`app.py`) integration, refresh metadata and deployment
Nguyen Thao Nhi	Project framing and user-story structure; GPQA data preparation; User Story 2

Trường	analysis, visualisation and verification
Santosh Pandit	User Story 3A (Tab 4): compute needed to reach a capability level — analysis, record and dumbbell charts, verification panel, layout alignment and README section
[Member 5, if applicable]	[TO COMPLETE]

Data Coverage and Refresh

Data coverage: through 09 September 2026
Last refreshed: generated when the data-preparation pipeline is run.

The **data coverage date** refers to the latest observation contained in the source data.

The **last refreshed date** should refer to when `prepared_data.py` most recently regenerated the processed datasets. It should not simply use the current time whenever `app.py` runs unless the app itself also refreshes the source data and rebuilds the processed files during that run.

A recommended implementation is for `prepared_data.py` to write the refresh timestamp to a small metadata file, for example:

`data/metadata.json`

and for `app.py` to read and display that timestamp.